

UNIVERSITÀ DEGLI STUDI DI NAPOLI “PARTHENOPE”
SCUOLA INTERDIPARTIMENTALE DELLE SCIENZE, DELL’INGEGNERIA E DELLA SALUTE
DIPARTIMENTO DI SCIENZE E TECNOLOGIE
CORSO DI LAUREA IN INFORMATICA



RELAZIONE DI ALGORITMI E STRUTTURE DATI

Traccia 1 — Esercizio 1
HashRBTree

DOCENTE
Prof. Ferone

STUDENTE
Raffaele Calcagno
0124001717

Anno Accademico 2021–2022

Capitolo 1

Descrizione del Problema

Il problema richiede la progettazione e l'implementazione di una struttura dati ibrida denominata **HashRBTree**, che combina le proprietà di una funzione di hash con quelle di un albero Red-Black (RBT).

L'obiettivo è realizzare una struttura in grado di memorizzare coppie (**chiave**, **valore**) di tipo stringa, supportando efficientemente le operazioni fondamentali di un dizionario:

- **Inserimento** di una coppia (**chiave**, **valore**);
- **Ricerca** del valore associato a una chiave;
- **Cancellazione** di una coppia (**chiave**, **valore**).

1.1 Approccio Adottato

La chiave viene trasformata in un intero tramite una funzione di hash, che diventa l'identificatore (**id**) del nodo all'interno dell'albero Red-Black. L'albero è quindi organizzato sui valori di **id**, garantendo ricerca, inserimento e cancellazione in tempo $O(\log n)$.

Le **collisioni** — situazioni in cui due chiavi diverse producono lo stesso valore di hash — vengono gestite tramite una lista concatenata associata a ciascun nodo. Ogni nodo dell'albero mantiene una `std::list` di coppie (**chiave**, **valore**), consentendo la coesistenza di più entry con lo stesso identificatore.

1.2 Motivazioni della Scelta

La combinazione di hash e Red-Black Tree offre un compromesso vantaggioso:

- La **funzione di hash** riduce lo spazio delle chiavi (stringhe arbitrarie) a un insieme di interi, rendendo possibile un ordinamento totale dei nodi;
- Il **Red-Black Tree** garantisce il bilanciamento automatico dell'albero, assicurando complessità logaritmica nel caso peggiore a differenza delle hash table classiche che degradano a $O(n)$ in caso di molte collisioni;
- La **lista di collisione** per nodo gestisce in modo semplice ed efficace le rare collisioni di hash senza richiedere strutture aggiuntive.

Capitolo 2

Descrizione delle Strutture Dati

2.1 KeyValue

La struttura `KeyValue` rappresenta la coppia chiave-valore memorizzata nella lista di collisione di ciascun nodo. È definita come segue:

```
1 struct KeyValue {  
2     string key;  
3     string value;  
4 };
```

Entrambi i campi sono di tipo `std::string`. La separazione della coppia in una struttura dedicata rende il codice più leggibile e facilita la gestione delle collisioni nella lista.

2.2 RBNode

`RBNode` rappresenta il nodo dell'albero Red-Black. Ogni nodo contiene:

- `int id` — identificatore numerico del nodo, ottenuto applicando la funzione di hash alla chiave inserita;
- `Color color` — colore del nodo, `RED` o `BLACK`, necessario per mantenere le proprietà dell'albero Red-Black;
- `std::list<KeyValue> items` — lista concatenata delle coppie (`chiave`, `valore`) che condividono lo stesso valore di hash (gestione delle collisioni);
- `RBNode* left`, `RBNode* right`, `RBNode* parent` — puntatori al figlio sinistro, figlio destro e nodo padre.

Il tipo enumerato `Color` è definito come:

```
1 enum Color { RED, BLACK };
```

Il costruttore di `RBNode` inizializza il colore a `RED` (ogni nuovo nodo inserito è sempre rosso), aggiunge la coppia (*chiave*, *valore*) alla lista `items` e imposta tutti i puntatori a `nullptr`:

```
1 RBNode(int id, const string& key, const string& value)
2     : id(id), color(RED), left(nullptr), right(nullptr),
3       parent(nullptr) {
4     items.push_back({key, value});
5 }
```

2.3 HashRBTree

`HashRBTree` è la classe principale che incapsula l'intera struttura dati. Mantiene internamente due puntatori:

- `RBNode* root` — puntatore alla radice dell'albero;
- `RBNode* NIL` — nodo sentinella condiviso da tutti i nodi foglia e dalla radice prima di qualsiasi inserimento.

2.3.1 Nodo Sentinella NIL

Invece di utilizzare `nullptr` per rappresentare i nodi foglia e i puntatori mancanti, si adotta un nodo sentinella `NIL` con le seguenti caratteristiche:

- colore `BLACK`;
- tutti i puntatori (`left`, `right`, `parent`) puntano a se stesso.

Questo approccio semplifica notevolmente l'implementazione di `insertFix` e `deleteFix`, eliminando la necessità di controllare `nullptr` ad ogni accesso ai campi del nodo: è sempre possibile accedere a `->color`, `->parent` ecc. senza rischio di dereferenziazione nulla.

2.3.2 Interfaccia Pubblica

La classe espone quattro metodi pubblici:

- `void insert(const string& key, const string& value)` — inserisce una coppia (chiave, valore);
- `void search(const string& key, const string& value)` — ricerca una chiave e stampa il valore associato;
- `void remove(const string& key, const string& value)` — rimuove una coppia (chiave, valore);
- `void print()` — stampa l'albero in forma grafica.

2.3.3 Metodi Privati

I metodi privati supportano le operazioni pubbliche:

- `hashFunction` — calcola l'hash della chiave;
- `leftRotate, rightRotate` — rotazioni per il bilanciamento;
- `insertFix, deleteFix` — ripristino delle proprietà RBT dopo inserimento e cancellazione;
- `transplant` — sostituzione di un sottoalbero con un altro;
- `minimum` — nodo con id minimo in un sottoalbero;
- `searchNode` — ricerca interna per id;
- `printHelper, destroyTree` — supporto a stampa e distruzione.

Capitolo 3

Formato dei Dati di Input e Output

3.1 Input

La struttura `HashRBTree` accetta in input coppie di stringhe (`chiave`, `valore`) tramite i metodi pubblici `insert`, `search` e `remove`. Non è previsto un formato di file esterno: le operazioni vengono invocate direttamente nel codice, come mostrato nel file `main.cpp`.

Ogni operazione richiede i seguenti parametri:

- `insert(key, value)` — due stringhe: la chiave da inserire e il valore associato;
- `search(key, value)` — la chiave da cercare (il parametro `value` è ignorato in questa operazione);
- `remove(key, value)` — la chiave della coppia da rimuovere (il parametro `value` è ignorato in questa operazione).

3.2 Output

L'output è prodotto su `stdout` tramite `std::cout`. Le operazioni producono i seguenti messaggi:

3.2.1 Search

- Se la chiave è presente:

```
1 Trovato: [chiave] -> valore
```

- Se la chiave non è presente:

```
1 Chiave "chiave" non trovata.
```

3.2.2 Remove

- Se la chiave non è presente:

```
1 Chiave "chiave" non trovata.
```

3.2.3 Print

Il metodo `print()` produce una rappresentazione grafica dell'albero con indentazione progressiva. Ogni nodo è stampato nel formato:

```
1 [id | colore] chiave:valore, chiave2:valore2
```

dove colore è R per RED e B per BLACK. I simboli `+-`, `'-` e `|` indicano la struttura gerarchica dell'albero. Un esempio di output:

```
1 +-- [1938937894 | B] banana:frutto giallo
2   +-- [253337143 | R] apple:frutto rosso
3     |   +-- [193491643 | B] fig:frutto mediterraneo
4     |   '--- [735886645 | B] elderberry:frutto viola
5     '--- [1986069970 | B] cherry:frutto rosso piccolo
6           '--- [2090176867 | R] date:frutto dolce
```

3.2.4 Albero Vuoto

Se `print()` viene invocato su un albero privo di elementi:

```
1 Albero vuoto.
```


Capitolo 4

Descrizione degli Algoritmi

4.1 Funzione di Hash

La funzione di hash adottata è l'algoritmo **djb2**, una funzione classica per stringhe proposta da Daniel J. Bernstein. La formula iterativa è:

$$h_0 = 5381, \quad h_{i+1} = h_i \cdot 33 + c_i \quad (4.1)$$

dove c_i è il valore ASCII dell' i -esimo carattere della chiave. Il risultato finale viene mascherato con `0x7FFFFFFF` per garantire che l'identificatore `id` sia sempre un intero non negativo.

L'algoritmo **djb2** offre una buona distribuzione dei valori di hash su stringhe di lunghezza variabile, minimizzando le collisioni nella pratica.

4.2 Inserimento

L'operazione di inserimento `insert(key, value)` si articola in due fasi:

1. **Calcolo dell'hash:** viene calcolato `id = hashFunction(key)` e si verifica tramite `searchNode(id)` se esiste già un nodo con quell'identificatore.
 - Se il nodo esiste (collisione), la coppia `(key, value)` viene aggiunta in coda alla lista `items` del nodo esistente. L'albero non viene modificato.
 - Se il nodo non esiste, si procede con l'inserimento BST.
2. **Inserimento BST:** il nuovo nodo viene inserito nell'albero navigando da sinistra a destra in base al valore di `id`. Il nodo è inizializzato con colore `RED`.

Dopo l'inserimento BST, viene invocato `insertFix` per ripristinare le proprietà del Red-Black Tree.

4.2.1 InsertFix

`insertFix` gestisce tre casi, applicati simmetricamente a seconda che il padre del nodo inserito sia figlio sinistro o destro del nonno:

- **Caso 1 — Zio RED:** il padre e lo zio vengono colorati **BLACK**, il nonno viene colorato **RED** e il controllo risale al nonno.
- **Caso 2 — Zio BLACK, nodo figlio destro:** viene eseguita una rotazione sinistra sul padre, trasformando la situazione nel Caso 3.
- **Caso 3 — Zio BLACK, nodo figlio sinistro:** il padre viene colorato **BLACK**, il nonno **RED**, e viene eseguita una rotazione destra sul nonno.

Al termine del ciclo, la radice viene forzata a **BLACK**.

4.3 Ricerca

L'operazione di ricerca `search(key, value)` si articola in due fasi:

1. **Ricerca nell'albero:** viene calcolato `id = hashFunction(key)` e si naviga l'albero tramite `searchNode(id)`, scendendo a sinistra se `id < node->id` e a destra altrimenti.
2. **Ricerca nella lista:** una volta trovato il nodo, si scorre la lista `items` confrontando la chiave esatta per risolvere eventuali collisioni.

Se il nodo non viene trovato oppure la chiave non è presente nella lista, viene stampato un messaggio di errore.

4.4 Cancellazione

L'operazione di cancellazione `remove(key, value)` opera su due livelli:

1. **Rimozione dalla lista:** viene cercata la coppia esatta (`key, value`) nella lista `items` del nodo. Se trovata, viene rimossa. Se la lista contiene ancora altri elementi, l'operazione termina qui e l'albero non viene modificato.

2. **Rimozione dall'albero:** se la lista è rimasta vuota, il nodo viene rimosso dall'albero secondo i tre casi classici del RBT:

- **Caso 1:** il nodo non ha figlio sinistro — il sottoalbero destro lo sostituisce tramite `transplant`.
- **Caso 2:** il nodo non ha figlio destro — il sottoalbero sinistro lo sostituisce tramite `transplant`.
- **Caso 3:** il nodo ha entrambi i figli — viene cercato il successore in-order (minimo del sottoalbero destro) tramite `minimum`, che prende il posto del nodo rimosso.

Se il nodo rimosso (o il suo successore nel Caso 3) aveva colore **BLACK**, viene invocato `deleteFix` per ripristinare le proprietà RBT.

4.4.1 DeleteFix

`deleteFix` gestisce quattro casi, applicati simmetricamente:

- **Caso 1 — Fratello RED:** rotazione sinistra sul padre e recoloring, che trasforma la situazione in uno dei casi seguenti.
- **Caso 2 — Fratello BLACK, entrambi i figli BLACK:** il fratello viene colorato RED e il controllo risale al padre.
- **Caso 3 — Fratello BLACK, figlio destro BLACK:** rotazione destra sul fratello e recoloring, che trasforma la situazione nel Caso 4.
- **Caso 4 — Fratello BLACK, figlio destro RED:** rotazione sinistra sul padre e recoloring. Il ciclo termina forzando `x = root`.

Al termine, il nodo corrente viene colorato **BLACK**.

4.5 Stampa

Il metodo `print()` invoca ricorsivamente `printHelper`, che visita l'albero in pre-order (radice, figlio sinistro, figlio destro) producendo una rappresentazione grafica con indentazione progressiva. Per ogni nodo vengono stampati l'identificatore, il colore e tutte le coppie presenti nella lista `items`.

Capitolo 5

Class Diagram

Di seguito viene presentato il diagramma delle classi della struttura `HashRBTree`, realizzato seguendo la notazione UML.

5.1 Struttura Generale

Il sistema è composto da tre elementi principali:

- la struttura `KeyValue`, che rappresenta la coppia chiave-valore;
- la classe `RBNode`, che rappresenta il nodo dell'albero;
- la classe `HashRBTree`, che incapsula l'intera struttura dati e le operazioni su di essa.

5.2 Diagramma UML

5.3 Relazioni tra le Classi

- **HashRBTree** ha una relazione di **composizione** con **RBNode**: i nodi sono creati e distrutti esclusivamente dalla classe `HashRBTree` tramite i metodi `insert` e `destroyTree`.
- **RBNode** ha una relazione di **aggregazione** con **KeyValue**: ogni nodo contiene una `std::list<KeyValue>` che può ospitare una o più coppie chiave-valore in caso di collisione.

- **RBNode** ha una relazione di **auto-associazione** tramite i puntatori **left**, **right** e **parent**, che collegano i nodi tra loro formando la struttura ad albero.
- Il tipo enumerato **Color** (**RED**, **BLACK**) è utilizzato da **RBNode** per mantenere le proprietà cromatiche del Red-Black Tree.

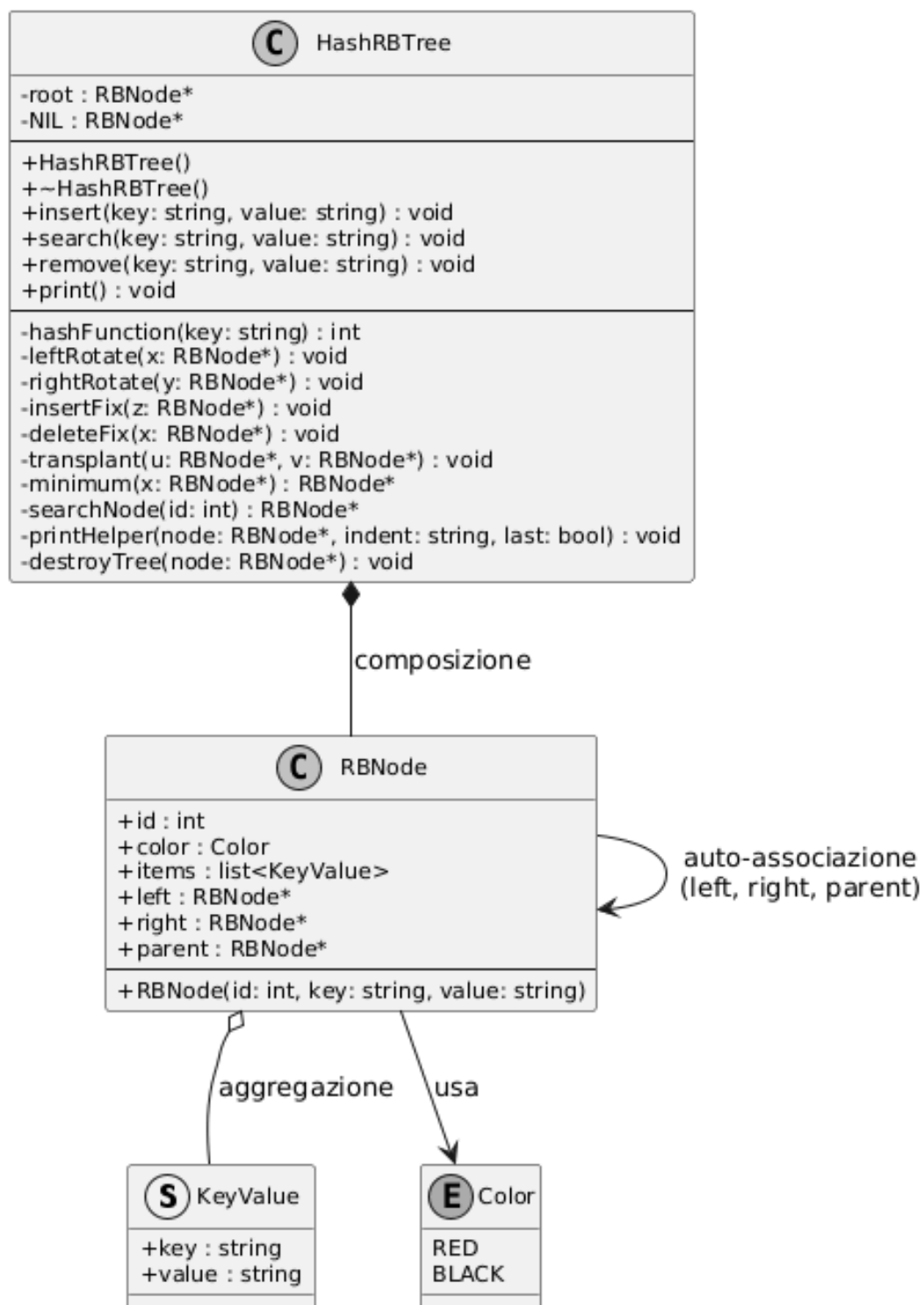


Figura 5.1: Class diagram UML di HashRBTree

Capitolo 6

Studio della Complessità

In questo capitolo viene analizzata la complessità temporale e spaziale delle operazioni principali della struttura `HashRBTree`.

6.1 Complessità Temporale

6.1.1 Funzione di Hash

La funzione `hashFunction` itera su tutti i caratteri della chiave eseguendo un numero costante di operazioni per ciascuno. La complessità è:

$$T_{hash} = O(m) \tag{6.1}$$

dove m è la lunghezza della chiave in caratteri.

6.1.2 Ricerca nell'Albero

Il metodo `searchNode` naviga l'albero Red-Black dall'alto verso il basso confrontando il valore `id` ad ogni nodo. Poiché l'albero è bilanciato per costruzione, l'altezza è garantita essere $O(\log n)$, dove n è il numero di nodi distinti nell'albero. La complessità è:

$$T_{searchNode} = O(\log n) \tag{6.2}$$

6.1.3 Ricerca nella Lista di Collisione

Una volta individuato il nodo tramite `searchNode`, il metodo `search` scorre la lista `items` per trovare la chiave esatta. Indicando con k il numero di coppie in collisione su quel nodo:

$$T_{search} = O(m + \log n + k) \quad (6.3)$$

Nel caso medio, con una buona funzione di hash, k è costante e la complessità si riduce a $O(m + \log n)$.

6.1.4 Inserimento

L'operazione `insert` esegue in sequenza:

1. Calcolo dell'hash: $O(m)$
2. Ricerca del nodo: $O(\log n)$
3. In caso di collisione, aggiunta in coda alla lista: $O(k)$
4. In caso di nuovo nodo, inserimento BST: $O(\log n)$
5. Ripristino proprietà RBT con `insertFix`: $O(\log n)$

La complessità complessiva è:

$$T_{insert} = O(m + \log n) \quad (6.4)$$

`insertFix` esegue al più $O(\log n)$ recoloring risalendo l'albero, e al più 2 rotazioni che sono operazioni $O(1)$.

6.1.5 Cancellazione

L'operazione `remove` esegue in sequenza:

1. Calcolo dell'hash: $O(m)$
2. Ricerca del nodo: $O(\log n)$
3. Rimozione dalla lista: $O(k)$

4. Se la lista è vuota, rimozione dall'albero: $O(\log n)$
5. Ripristino proprietà RBT con `deleteFix`: $O(\log n)$

La complessità complessiva è:

$$T_{remove} = O(m + \log n) \quad (6.5)$$

`deleteFix` esegue al più $O(\log n)$ recoloring e al più 3 rotazioni.

6.1.6 Riepilogo

Operazione	Caso Medio	Caso Peggior
<code>hashFunction</code>	$O(m)$	$O(m)$
<code>search</code>	$O(m + \log n)$	$O(m + \log n + k)$
<code>insert</code>	$O(m + \log n)$	$O(m + \log n + k)$
<code>remove</code>	$O(m + \log n)$	$O(m + \log n + k)$

dove n è il numero di nodi nell'albero, m è la lunghezza della chiave e k è il numero di collisioni sul nodo.

6.2 Complessità Spaziale

La struttura occupa in memoria:

- $O(n)$ per i nodi dell'albero, dove n è il numero di chiavi distinte per hash;
- $O(n + p)$ considerando anche le coppie nelle liste di collisione, dove p è il numero totale di coppie inserite;
- $O(\log n)$ per lo stack delle chiamate ricorsive di `printHelper` e `destroyTree`.

La complessità spaziale complessiva è:

$$S = O(n + p) \quad (6.6)$$

Capitolo 7

Test e Risultati

In questo capitolo vengono presentati i test eseguiti sulla struttura `HashRBTree` per verificare la correttezza delle operazioni di inserimento, ricerca, cancellazione e stampa.

7.1 Ambiente di Test

I test sono stati eseguiti nel seguente ambiente:

- **Sistema operativo:** Linux Mint
- **Compilatore:** GCC 13.3.0
- **Standard:** C++17
- **Build system:** CMake 3.10

7.2 Test 1 — Inserimento e Stampa

Vengono inserite sette coppie chiave-valore e viene verificata la struttura dell'albero tramite `print()`.

```
1 tree.insert("apple", "frutto_rosso");  
2 tree.insert("banana", "frutto_giallo");  
3 tree.insert("cherry", "frutto_rosso_piccolo");  
4 tree.insert("date", "frutto_dolce");  
5 tree.insert("elderberry", "frutto_viola");  
6 tree.insert("fig", "frutto_mediterraneo");  
7 tree.insert("grape", "frutto_a_grappolo");
```

Output ottenuto:

```
1 +-- [1938937894 | B] banana:frutto giallo
2   +-- [253337143 | R] apple:frutto rosso
3   |   +-- [193491643 | B] fig:frutto mediterraneo
4   |   '-- [735886645 | B] elderberry:frutto viola
5   |       +-- [260508340 | R] grape:frutto a grappolo
6   '-- [1986069970 | B] cherry:frutto rosso piccolo
7       '-- [2090176867 | R] date:frutto dolce
```

Verifica: l'albero rispetta tutte le proprietà del Red-Black Tree:

- la radice è BLACK;
- non esistono due nodi RED consecutivi;
- il black-height è uniforme su tutti i percorsi dalla radice alle foglie.

7.3 Test 2 — Ricerca

```
1 tree.search("apple", "");
2 tree.search("banana", "");
3 tree.search("mango", "");
```

Output ottenuto:

```
1 Trovato: apple -> frutto rosso
2 Trovato: banana -> frutto giallo
3 Chiave "mango" non trovata.
```

Verifica: le chiavi presenti vengono trovate correttamente; la ricerca di una chiave assente restituisce il messaggio di errore senza crash.

7.4 Test 3 — Gestione delle Collisioni

Vengono inserite due chiavi aggiuntive per verificare il comportamento in caso di collisioni hash.

```
1 tree.insert("listen", "ascoltare");
2 tree.insert("silent", "silenzioso");
```

Verifica: la funzione djb2 è sensibile all'ordine dei caratteri, quindi `listen` e `silent` producono hash distinti e vengono inseriti come nodi separati. L'albero si ribilancia correttamente dopo entrambi gli inserimenti.

7.5 Test 4 — Cancellazione

```
1 tree.remove("banana", "");
2 tree.remove("mango", "");
```

Output ottenuto:

```
1 Chiave "mango" non trovata.
2
3 +-- [1986069970 | B] cherry:frutto rosso piccolo
4   +-- [253337143 | R] apple:frutto rosso
5     |   +-- [193491643 | B] fig:frutto mediterraneo
6       |   |   +-- [192495636 | R] listen:ascoltare
7         |   |   '-- [466175796 | B] silent:silenzioso
8         |   +-- [260508340 | R] grape:frutto a grappolo
9         |   '-- [735886645 | R] elderberry:frutto viola
10        '-- [2090176867 | B] date:frutto dolce
```

Verifica: la rimozione di **banana** (nodo radice) viene gestita correttamente tramite il successore in-order; l'albero rimane bilanciato. La rimozione di **mango** (chiave assente) produce il messaggio di errore senza alterare la struttura.

7.6 Test 5 — Cancellazioni Multiple

```
1 tree.remove("apple", "");
2 tree.remove("cherry", "");
```

Output ottenuto:

```
1 +-- [260508340 | B] grape:frutto a grappolo
2   +-- [193491643 | B] fig:frutto mediterraneo
3     |   +-- [192495636 | R] listen:ascoltare
4       '-- [735886645 | R] elderberry:frutto viola
5         +-- [466175796 | B] silent:silenzioso
6         '-- [2090176867 | B] date:frutto dolce
```

Verifica: dopo tre cancellazioni consecutive l'albero rimane correttamente bilanciato, confermando il corretto funzionamento di `deleteFix` in scenari di stress.

7.7 Considerazioni Finali

I test confermano che la struttura `HashRBTree` si comporta correttamente in tutti gli scenari verificati:

- le proprietà del Red-Black Tree vengono mantenute dopo ogni inserimento e cancellazione;
- la gestione delle collisioni tramite lista funziona correttamente;
- i casi limite (chiave assente, albero con un solo nodo, cancellazione della radice) sono gestiti senza errori.